Scope: FinTrack

A personal money tracker for one person. You log what you spend, put it in a category, and the app tells you where your money actually went this month. It starts as the smallest version you would genuinely use, and grows from there.

Build approach: Skateboard (ship the thinnest usable whole first, then grow it release by release, shippable at every step). **Workflow:** Alpha (after `/develop`, run `/check verify` on the real app; no separate test suite or fresh model review unless a feature asks for it). The project default level of rigor. `/architect` is the recommended first stop for a feature with a real decision, but skippable when you already know the build. Any feature can carry its own tag (e.g. `· GA`) to do more or less.

These are recommendations to keep your build orderly, not requirements. Skip anything that does not fit: if you already know how to build a feature, use `/develop` and skip `/architect`. You decide when a feature is `done`.

At a glance

#	Feature	Phase	Status
1	Stack and architecture	Foundation	planned
2	Coding standards and tooling	Foundation	planned
3	Data model	Foundation	planned
4	Design system and UI foundation	Foundation	planned
5	Sign in and your account	Release 1	planned
6	Log a spend	Release 1	planned
7	This month's transactions	Release 1	planned
8	Where your money went	Release 1	planned
9	Categories you manage	Release 2	planned
10	Search and filter your history	Release 2	planned
11	Export and backup	Release 2	planned
12	Error monitoring	Release 2	planned
13	Budgets per category	Release 3	planned
14	Income and money coming in	Release 4	planned
15	Recurring bills and subscriptions	Release 4	planned
16	Trends across months	Release 5	planned
17	Accounts and balances	Release 5	planned

#	Feature	Phase	Status
18	Receipt photo capture	Release 6	planned
19	Bank connection	Release 6	planned
20	Public landing page	Release 6	planned

Foundations

1. Stack and architecture · needs a decision · Beta

Decide what this app is built with, then scaffold a project that actually runs, so every later release stands on real structure rather than guesses. Your light preferences about how code should be laid out belong here too. **Done when:** the stack choice is written down in a spec, the scaffold boots locally, and a production build passes. (basis: nothing tooling related should run before the stack decision, so this is the one place tools get chosen)

- [ ] Decide the stack (spec): `/architect stack & architecture`
- [ ] Scaffold from the decision: `/develop stack & architecture`

2. Coding standards and tooling

Capture the real conventions from the scaffolded project, then install the lint, format, and check tooling that all later code has to follow. **Done when:** root `AGENTS.md` describes the real stack and conventions, and the lint and format commands run clean.

- [ ] Capture conventions and tooling choices: `/audit`
- [ ] Install the tooling: `/develop tooling`

3. Data model · needs a decision · Beta

The core things this app stores: you, your transactions, your categories, and how they connect. Money amounts and dates live here. **Done when:** the model holds both a spend and an income entry against a category and a date, amounts are stored exactly rather than as approximate decimals, and the later releases (budgets, repeating bills, several accounts) fit without a breaking change. (basis: a wrong data model is the most expensive thing to redo, and binary floating point is the classic way money math goes quietly wrong)

- [ ] Design it (spec): `/architect data model`

4. Design system and UI foundation · needs a decision

The visual language and the base components every screen reuses: type, color, spacing, forms, buttons, and the empty and error states. **Done when:** `design.md` covers type, color, spacing, and components, and the base components work with a keyboard alone and read correctly to a screen reader. (basis: WCAG 2.2, and keyboard first entry also makes daily logging less annoying)

- [ ] Design it (spec): `/architect design system & UI foundation`

Release 1: the thinnest usable whole

The smallest version you would genuinely open every day: one account, log a spend, see where the month went. Nothing else. (basis: smallest usable product, so real use tells you what to build next)

5. Sign in and your account · needs a decision · Beta

One account that is yours, so your data follows you from laptop to phone. No sharing, no other people. **Done when:** you can create the account, sign in, stay signed in between visits, sign out, and get back in if you forget how; nothing readable without being signed in. (basis: OWASP authentication guidance; bumped to Beta because this one guards everything else)

- [ ] Design it (spec): `/architect sign in and your account`

6. Log a spend

The screen you will use more than any other: amount, date, category, optional note, saved in seconds. **Done when:** you can add a spend in a few seconds from the main screen, it saves to your account, and a missing amount or category is caught before saving rather than after.

- [ ] Build it: `/develop log a spend`

7. This month's transactions

A plain list of what you logged this month, so you can spot a mistake and fix it. **Done when:** the month's entries show newest first with a running total, you can edit or delete any one of them, and an empty month says so plainly instead of looking broken.

- [ ] Build it: `/develop this month's transactions`

8. Where your money went · needs a decision

The screen that answers your actual question: total spent this month, split by category, biggest first. **Done when:** the month total and the split by category are correct against the logged entries, the biggest category is obvious at a glance, and a month with no data reads as empty rather than broken. (basis: this is the core value screen and how you show the split is a real design call, so it earns a spec)

- [ ] Design it (spec): `/architect where your money went`

Release 2: make it yours, and keep it safe

9. Categories you manage

Rename, add, and hide categories so the breakdown matches how you actually think about your money. **Done when:** you can add, rename, and hide a category, and existing entries still point at the right one after a rename.

- [ ] Build it: `/develop categories you manage`

10. Search and filter your history

Chase down one specific charge, or look at a single category over a stretch of time. **Done when:** you can filter by category, by date range, and by text in the note, and the filters survive a page reload.

- [ ] Build it: `/develop search and filter your history`

11. Export and backup

Take everything you logged out of the app as a file, so months of typing are never trapped in here. **Done when:** one action downloads every transaction and category in a format a spreadsheet opens, and the file matches what the app shows.

- [ ] Build it: `/develop export and backup`

12. Error monitoring · needs a decision

Find out when something broke instead of silently losing an entry. **Done when:** an error in the running app reaches somewhere you will actually see it, with enough detail to act on, and no money amounts or personal detail travel with it.

- [ ] Design it (spec): `/architect error monitoring`

Release 3: control

13. Budgets per category · needs a decision

Set a monthly cap per category and see what is left, so the breakdown turns into a decision instead of a report. **Done when:** you can set a cap per category, see spent against remaining for the current month, and see clearly when a category has gone over.

- [ ] Design it (spec): `/architect budgets per category`

Release 4: the whole money picture

14. Income and money coming in

Log salary and anything else coming in, so the app can show what is left rather than only what left. **Done when:** income entries are logged and kept apart from spending, and a month shows money in, money out, and the difference.

- [ ] Build it: `/develop income and money coming in`

15. Recurring bills and subscriptions · needs a decision

Rent, subscriptions, and anything else that repeats, so you stop typing them and stop forgetting them. **Done when:** a repeating entry is defined once, shows up in the right months without more typing, and can be paused or ended without rewriting past months.

- [ ] Design it (spec): `/architect recurring bills and subscriptions`

Release 5: perspective over time

16. Trends across months · needs a decision

How your spending moves from month to month, and which categories moved the most. **Done when:** you can compare the last several months overall and per category, and the biggest movers are called out

rather than left for you to spot.

- [ ] Design it (spec): `/architect trends across months`

17. Accounts and balances · needs a decision

Cash, card, bank, savings, each with its own balance, rolled up into one number. **Done when:** an entry belongs to an account, each account shows a balance you trust, and the total across accounts is one clear number.

- [ ] Design it (spec): `/architect accounts and balances`

Release 6: faster entry, and a public face

18. Receipt photo capture · needs a decision

Snap a receipt and let the app fill in amount, date, and merchant, with you correcting whatever it got wrong. **Done when:** a photo produces a draft entry you confirm or fix before it saves, and a failed read falls back to normal manual entry rather than losing the receipt.

- [ ] Design it (spec): `/architect receipt photo capture`

19. Bank connection · needs a decision · GA

Transactions arrive on their own, so logging turns into reviewing. **Done when:** transactions come in automatically, duplicates against your manual entries are caught, you can disconnect and delete everything that was pulled, and access goes through a regulated bank interface rather than you handing over your banking password. (basis: open banking rules such as PSD2 exist precisely so apps stop asking for bank credentials; bumped to GA because this is the one feature holding a live financial connection)

- [ ] Design it (spec): `/architect bank connection`

20. Public landing page · needs a decision

One page describing the app for other people, for whenever you want to show it to someone. **Done when:** the page explains what the app does, loads fast, and carries the title, description, social card, and structured data a search engine expects. (basis: search engine guidance on page metadata and structured data; kept as a single page so the rest of the app stays private and login walled)

- [ ] Design it (spec): `/architect public landing page`

Deferred

Out of scope for the current build pass, kept so the plan stays honest.

- **Sharing with a partner**: shared household spending and categories · needs a decision
- **Several currencies**: amounts in more than one currency, with rates · needs a decision
- **Savings goals**: put money aside toward a target and track it · needs a decision
- **Install on your phone**: keep it on the home screen and log offline · needs a decision
- **Written insights**: the app tells you what changed and why it matters · needs a decision

Legend

The decision box. Every feature carries exactly one, the sub task whose label ends with `(spec)`. Its wording varies (`Design it (spec)` normally, `Decide the stack (spec)` on Stack and architecture), so skills find it by that `(spec)` suffix, never by an exact label. Every other box is an execution box and `/architect` never ticks one.

- **Next step** = the first unticked box (always a command or a tracked milestone).
- **needs a decision** = run `/architect` first; otherwise straight to `/develop` (or `/audit` for standards and tooling). The tag drops once the spec is captured.
- **Atomic build tasks live in the spec's `## Build plan`, not here**: this file carries only the milestone rollup that `/architect` fills in.
- **Status** `planned` → `in-progress` → `done`, plus `existing` (predates this workflow) and `dropped` (removed from scope, kept for history).
- **Approach tag** beside a heading (e.g. `· Facade`) overrides the project default for that one feature; no tag means it inherits.
- **Workflow tier tag** beside a heading (e.g. `· GA`, `· Beta`) sets that one feature's rigor above or below the project default; no tag inherits Alpha.
- **Workflow** (header line) is the project default, what runs after `/develop`: **Prototype** = nothing; **Alpha** = `/check verify`; **Beta** = `/check verify` then `/test`; **GA** = adds a fresh model `/check review` then `/document`.
- **Pointer line** (`spec <n> · code in <path>`): the spec link added by `/architect`, the code path by `/develop`.

References

Project sources

- None yet. This is the first pass on an empty project, so nothing here comes from existing code or an `AGENTS.md`. Once `/audit` runs, later scope passes can cite the real project.

Practices and standards

- Foundations before features: stack, data model, and design system are laid before any release slice, cheapest first.
- Smallest usable product: Release 1 is a whole thing you would use, not a half built chassis.
- The data model is the costliest thing to redo, so it gets its own spec and its own decision.
- Money is stored exactly, never as binary floating point.
- WCAG for keyboard and screen reader access, folded into the design system rather than bolted on later.
- OWASP guidance for sign in and for handling your personal financial data.
- Regulated open banking access rather than credential sharing, for the future bank connection.
- Search engine guidance on page metadata and structured data, for the one public page.

Links (fetched and confirmed once, for you to follow if you want the source)

- Smallest usable product: Making sense of MVP, Henrik Kniberg https://blog.crisp.se/2016/01/25/henrikkniberg/making-sense-of-mvp
- Money storage: Money pattern, Martin Fowler https://martinfowler.com/eaaCatalog/money.html

- Accessibility: WCAG 2.2, W3C https://www.w3.org/TR/WCAG22/
- Security baseline: OWASP Application Security Verification Standard https://owasp.org/www-project-application-security-verification-standard/
- Security baseline: OWASP Top Ten https://owasp.org/www-project-top-ten/
- Sign in: OWASP Authentication Cheat Sheet https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
- Bank access: the revised Payment Services Directive (PSD2), European Central Bank https://www.ecb.europa.eu/press/intro/mip-online/2018/html/1803_revisedpsd.en.html
- Landing page: Intro to how structured data markup works, Google Search Central https://developers.google.com/search/docs/appearance/structured-data/intro-structured-data

Not verified, so not linked: the walking skeleton idea is usually credited to Alistair Cockburn, but no primary page could be confirmed, so it stays here as a named practice only.